JARVIS OS

AI Operating System

PRODUCT REQUIREMENT DOCUMENT

Version 2.0 | Comprehensive Edition

Project Name	JARVIS OS (Software Version)
Version	2.0
Status	Active Development
Date	February 2026
Tech Stack	FastAPI + React (Vite) + Gemini API
Owner	JARVIS Engineering Team
Confidentiality	Internal / Restricted

CONFIDENTIAL — FOR INTERNAL USE ONLY

1. Executive Summary

JARVIS OS (Just A Rather Very Intelligent System Operating System) is a next-generation, multi-agent AI assistant platform engineered to function as a personal AI Operating System. Unlike conventional chatbots, JARVIS is designed to reason across complex tasks, maintain persistent long-term memory, coordinate specialized AI agents, execute real-world system commands, and communicate naturally through both voice and text interfaces.

This document outlines the full product scope, architecture, requirements, acceptance criteria, risk analysis, and delivery roadmap for JARVIS OS v2.0.

Mission Statement
To empower developers, students, and AI enthusiasts with a personal AI OS that can think, remember, speak,
and act — transforming how individuals interact with their computing environment through intelligent automation.

2. Problem Statement

Current AI assistant solutions have fundamental limitations that prevent them from being truly useful in real-world workflows:

Problem	Impact
No persistent memory — every conversation starts fresh	Users must repeat context constantly; no personalization
Single-model, single-task architecture	Cannot parallelize reasoning or delegate to specialized agents
No system-level control	Cannot automate OS tasks, files, or applications
Limited voice interaction pipelines	Speech feels disconnected from reasoning; high latency
No extensible agent framework	Developers cannot plug in custom agents or tools
Poor context management in long sessions	Responses degrade over extended conversations

3. Product Vision & Goals

3.1 Vision Statement

Build a real-time, multi-agent AI system that can communicate via voice and text, reason across complex goals, store and retrieve long-term memory, perform system-level tasks, and serve as a platform for future hardware integration — functioning as a true AI Operating System for the individual.

3.2 Strategic Goals

Goal	Description	Success Indicator
Intelligent Reasoning	Multi-agent brain for decision making and task delegation	95%+ intent routing accuracy
Persistent Memory	Long-term user context via vector embeddings	Recall accuracy > 90% over 30 days
Voice-First Interface	Low-latency voice I/O as primary interaction mode	End-to-end voice response < 2.5s
System Automation	Native OS task execution and app control	100% safe sandboxed execution
Developer Platform	Extensible agent/plugin architecture	Custom agents deployable in < 30 min
Scalable Deployment	Dockerized, API-first, multi-environment	Zero-downtime deploys, Docker-ready

4. Target Users & Personas

4.1 User Segments

Persona	Profile	Primary Need
Python Developer	Mid-level dev building AI pipelines, familiar with APIs	Extensible agent framework & clean APIs
CS Student	Learning AI/ML, wants hands-on architecture examples	Well-documented code, modular design
AI Enthusiast	Non-developer, wants personal AI assistant	Easy setup, voice interface, natural UX
Power Automation User	Knowledge worker who automates repetitive tasks	File ops, browser automation, scheduling

Researcher	AI/ML researcher evaluating multi-agent systems	RAG pipeline, memory architecture access

4.2 User Stories

As a Python Developer, I want to register a new custom agent via a plugin interface so that I can extend JARVIS without modifying core code.

As a Student, I want to view the agent routing decisions in a debug panel so that I can understand how JARVIS selects agents for each task.

As an AI Enthusiast, I want to speak a command and hear a response in under 3 seconds so that interaction feels natural and responsive.

As an Automation User, I want to schedule daily reminders and file backups so that JARVIS manages my workflow automatically.

As a Researcher, I want to query the vector memory store directly so that I can inspect what JARVIS has learned about my usage patterns.

5. Core Features — Phased Delivery

PHASE 1 — MVP: Core Chat System

Phase 1 Scope — Week 1–2
Delivery: Functional real-time chat interface with streaming AI responses
Success: User can send a message and receive a streamed Gemini response via WebSocket

5.1.1 Backend (FastAPI)

- Async REST API with FastAPI + Uvicorn
- WebSocket endpoint for real-time streaming responses
- Gemini API integration (gemini-pro model) with retry logic
- Session management with UUID-based session tokens
- Short-term in-memory conversation history (last N turns configurable)
- Structured JSON logging with request/response tracing
- Global error handler with typed error responses
- CORS middleware with configurable allow-list
- Environment-based configuration (.env via pydantic-settings)

5.1.2 Frontend (React + Vite)

- Responsive single-page chat UI
- Real-time streaming message rendering (token-by-token)
- Message history panel with timestamps

- Loading/typing indicators
- Dark and light mode toggle
- Mobile-responsive layout
- Error toasts for API failures

5.1.3 Acceptance Criteria — Phase 1

Criteria	Pass Condition
Message delivery	User message sent and response received within 2 seconds (non-streamed)
Streaming	Tokens render progressively; no blank screen during generation
Session persistence	Conversation context maintained across 20+ turns in session
Error recovery	API timeout retried 3x before showing user error
WebSocket stability	Connection auto-reconnects within 2s of drop

PHASE 2 — Advanced Brain: Multi-Agent Architecture

Phase 2 Scope — Week 3
Delivery: Central intelligence layer that routes user intent to specialized agents
Success: 5 core agents operational; routing accuracy > 90% on test suite

5.2.1 Central Brain

- Intent Classification Engine — rule-based + LLM-assisted hybrid
- Agent Registry — dynamic agent discovery and registration
- Task Planner — decomposes multi-step requests into sub-tasks
- Agent Orchestrator — parallel/sequential agent execution
- Response Aggregator — merges multi-agent outputs into coherent response
- Fallback Handler — unknown intents routed to raw Gemini LLM

5.2.2 Core Agents

Agent	Responsibilities	Inputs / Outputs
Chat Agent	General conversation, Q&A, summarization, creative writing	Text prompt → Text response
System Control Agent	File operations, process control, OS commands (sandboxed)	Command string → Execution result

Web Agent	Web search, page scraping, URL summarization	Query/URL → Structured data or summary
File Agent	Read, write, move, rename, organize files	File path + operation → Success/Error + result
Memory Agent	Store, retrieve, and manage user memory entries	Query/content → Retrieved context or confirmation

5.2.3 Agent Execution Flow

User Input → Intent Detection → Task Planning → Agent Selection → Parallel/Sequential Execution → Response Aggregation → Output

5.2.4 Acceptance Criteria — Phase 2

Criteria	Pass Condition
Intent routing accuracy	> 90% correct agent selected on 100-item test suite
Multi-agent coordination	Parallel agents complete without race conditions
Fallback behavior	Unknown intents return graceful LLM response, never crash
Agent isolation	Agent failure does not cascade to other agents
Plugin registration	New agent registered and callable without restart

PHASE 3 — Voice System

Phase 3 Scope — Week 4 (Part 1)
Delivery: Fully functional voice-in, voice-out interaction pipeline
Success: End-to-end voice interaction latency under 3 seconds on local hardware

5.3.1 Voice Pipeline Components

- Browser Microphone Capture via WebAudio API + MediaRecorder
- Speech-to-Text (STT): OpenAI Whisper (whisper-base/small model, local inference)
- Text-to-Speech (TTS): Coqui TTS (VITS model for natural prosody)
- Streaming audio playback — TTS chunks streamed as generated
- Push-to-talk mode + always-on wake word detection (optional Phase 3.5)
- Voice activity detection (VAD) to auto-detect speech start/end
- Noise suppression preprocessing before STT

5.3.2 Voice Configuration Options

Setting	Options / Default
STT Model	whisper-tiny / whisper-base (default) / whisper-small
TTS Voice	VITS multi-speaker: en_US male (default), en_US female, custom
Input Mode	Push-to-talk (default) / Always-on VAD
Language	English (default), Multilingual (Phase 4 extension)
Audio Quality	16kHz mono (STT) / 22kHz stereo (TTS)

5.3.3 Acceptance Criteria — Phase 3

Criteria	Pass Condition
STT accuracy	> 95% word accuracy on clear speech, standard accent
TTS naturalness	MOS score > 3.5 in user testing (n=10)
End-to-end latency	< 3 seconds from speech end to audio response start
VAD accuracy	< 5% false positive activations in quiet environment
Browser support	Works on Chrome, Firefox, Edge (latest 2 versions)

PHASE 4 — Long-Term Memory

Phase 4 Scope — Week 4 (Part 2)
Delivery: Persistent vector memory with RAG-powered retrieval
Success: JARVIS recalls user-specific facts across sessions with > 85% relevance

5.4.1 Memory Architecture

- Vector Store: ChromaDB (local, persistent, Docker-mounted volume)
- Embedding Model: Gemini Embedding API / sentence-transformers (fallback)
- Memory Types: Episodic (events), Semantic (facts), Procedural (preferences)
- RAG Pipeline: Top-K retrieval (k=5 default) injected into LLM context window
- Memory Manager Agent: Decides what to store, when to update, what to forget
- User behavior profiling: Frequency analysis of commands, topics, preferences
- Memory expiry: Configurable TTL with relevance-weighted retention
- Export/Import: JSON export of full memory store for backup and portability

5.4.2 Memory Schema

Field	Description
memory_id	UUID — unique identifier for each memory entry
type	episodic | semantic | procedural
content	Raw text content of the memory
embedding	Vector embedding (float array)
timestamp	ISO 8601 creation/update timestamp
source	Which agent or interaction created this memory
relevance_score	Float 0–1, updated via usage frequency
ttl_days	Days until expiry (-1 = permanent)
tags	List of topic/entity tags for filtering

5.4.3 Acceptance Criteria — Phase 4

Criteria	Pass Condition
Cross-session recall	Facts mentioned in session 1 retrieved accurately in session 5
Retrieval relevance	Top-5 retrieved memories are relevant > 85% of the time
Write latency	Memory stored in < 200ms after conversation turn
Data portability	Memory exported to valid JSON in < 5 seconds
Privacy control	User can delete all memories via single command

PHASE 5 — Automation System

Phase 5 Scope — Week 5
Delivery: Safe, sandboxed automation layer for OS and browser control
Success: Scheduled tasks execute reliably; no unintended system modifications

5.5.1 Automation Capabilities

- File System Operations: create, read, move, rename, delete, zip/unzip
- Application Launcher: open/close apps by name
- Browser Automation: Playwright-based web scraping, form filling, navigation
- Task Scheduler: cron-like scheduling with natural language time parsing
- Reminder System: time-based and event-based alerts with voice/text notification
- Clipboard Management: read/write system clipboard
- Screenshot Capture: full screen or region, saved to file

- System Monitoring: CPU, RAM, disk usage reporting

5.5.2 Security Sandbox Model

All automation commands are executed within a strict permission model:

- Allowlist-based file system access (user-defined safe directories only)
- No root/admin execution; all commands run as current user
- Dry-run mode: preview destructive operations before execution
- Full audit log of every executed command with timestamp and agent
- Emergency kill switch: single command to halt all automation immediately
- Rate limiting: max 10 automation commands per minute to prevent runaway loops

5.5.3 Acceptance Criteria — Phase 5

Criteria	Pass Condition
File operation safety	No file deletions outside allowed directory; blocked with error
Scheduler reliability	Scheduled tasks execute within 10 seconds of target time
Browser automation	Playwright scripts complete without memory leaks
Audit logging	Every command logged with timestamp, agent, result
Kill switch	All automation halted within 2 seconds of kill command

6. Functional Requirements

6.1 Backend Requirements

Requirement ID	Requirement
BE-01	FastAPI async endpoints for all API routes; no synchronous blocking calls
BE-02	WebSocket server supporting up to 50 concurrent connections
BE-03	Gemini API client with exponential backoff retry (3 attempts, 2/4/8s delays)
BE-04	Modular agent loader — agents discovered automatically from /agents directory
BE-05	Centralized exception handler returning typed JSON error objects
BE-06	Structured JSON logging to stdout; configurable log level via ENV

BE-07	Health check endpoint GET /health returning system status JSON
BE-08	Rate limiter: 60 requests/minute per IP (configurable)
BE-09	Request validation via Pydantic v2 models for all inputs
BE-10	Async background task queue for non-blocking agent execution

6.2 Frontend Requirements

Requirement ID	Requirement
FE-01	React 18 + Vite 5 SPA with TypeScript strict mode enabled
FE-02	WebSocket client with auto-reconnect and exponential backoff
FE-03	Streaming message rendering: tokens append in real-time
FE-04	Voice control panel: microphone toggle, waveform visualizer, status indicator
FE-05	Chat history persisted to localStorage with 100-message rolling window
FE-06	Responsive design: mobile (320px) through 4K (3840px) displays
FE-07	Dark/light theme with system preference detection
FE-08	Agent activity panel: shows which agent is processing each request
FE-09	Memory browser: view, search, and delete stored memories
FE-10	Settings panel: configure voice, model, theme, and memory preferences

6.3 AI Brain Requirements

Requirement ID	Requirement
AI-01	Intent classifier with confidence score output; low-confidence routes to LLM
AI-02	Agent registry with hot-reload support (no restart needed)
AI-03	Task planner capable of decomposing into ≥ 5 sequential sub-tasks
AI-04	Parallel agent execution via asyncio.gather with configurable timeout

AI-05	Memory retrieval injected into LLM system prompt (top-5 results)
AI-06	Agent output schema validation before passing to response aggregator
AI-07	Per-agent token budget limits to prevent context overflow
AI-08	Conversation summarization when context exceeds token limit

7. Non-Functional Requirements

7.1 Performance

Metric	Target
Chat response time (non-streamed)	< 2 seconds (p95)
Streaming first-token latency	< 500ms
Voice end-to-end latency	< 3 seconds
Memory write latency	< 200ms
Memory retrieval latency	< 100ms
WebSocket throughput	50 concurrent connections without degradation
Agent routing decision time	< 150ms
Frontend initial load time	< 2 seconds on 3G connection

7.2 Security Requirements

Requirement	Implementation
API key management	All secrets in .env; never committed; loaded via pydantic-settings
CORS policy	Allowlist of permitted origins; wildcard (*) forbidden in production
Input sanitization	All user inputs validated via Pydantic; HTML/script stripped
Authentication	JWT-based auth for API endpoints (Phase 2+); API key for local dev
Automation sandboxing	File ops restricted to USER_ALLOWED_PATHS; no sudo execution
Audit logging	All automation commands logged with user ID, timestamp, result

Dependency security	Weekly dependency scan via pip-audit / npm audit in CI
Rate limiting	60 req/min per IP; separate limits for voice and automation endpoints
Data encryption	Memory store encrypted at rest (AES-256 via SQLCipher for ChromaDB)
HTTPS enforcement	TLS required for all production deployments; HTTP redirect to HTTPS

7.3 Reliability & Availability

- System uptime target: 99.5% for local deployment
- Automatic recovery from Gemini API outages via cached fallback responses
- WebSocket reconnection within 2 seconds of connection drop
- ChromaDB persistence survives application restart without data loss
- Scheduler jobs survive application restart (persisted to SQLite)

7.4 Scalability

- Horizontal scaling: multiple FastAPI instances behind nginx load balancer
- Stateless API design — no server-side session state (sessions in Redis for scale)
- Docker + Docker Compose for single-node; Kubernetes manifests for multi-node
- Database connection pooling for all persistent stores
- ChromaDB supports sharding for memory stores > 1M entries

7.5 Maintainability

- Code coverage minimum: 80% unit tests, 60% integration tests
- All public functions and classes documented with docstrings
- API documentation auto-generated via FastAPI OpenAPI spec at /docs
- Changelog maintained (CHANGELOG.md) for every release
- Linting: Ruff (Python), ESLint (TypeScript); enforced in CI

8. Technical Architecture

8.1 High-Level Architecture

The JARVIS OS architecture follows a layered, event-driven design with clear separation between the user interface, orchestration layer, agent execution layer, and storage.

Architecture Flow
User (Voice / Text)
↓
React Frontend (WebSocket Client)
↓
FastAPI WebSocket Server
↓
Central Brain (Intent Classifier → Task Planner → Agent Orchestrator)
↓ ↓ ↓ ↓ ↓
Chat Agent | Web Agent | File Agent | System Agent | Memory Agent
↓
Tools Layer (Whisper, Coqui, Playwright, ChromaDB, OS APIs)
↓
Gemini API (External LLM)

8.2 Technology Stack

Layer	Technology	Purpose
Backend Framework	FastAPI 0.110+ / Uvicorn	Async API server, WebSocket support
Frontend	React 18 + Vite 5 + TypeScript	Interactive chat and voice UI
LLM Provider	Google Gemini API (gemini-pro)	Core AI reasoning and text generation
STT	OpenAI Whisper (local inference)	Speech-to-text transcription
TTS	Coqui TTS (VITS model)	Text-to-speech synthesis
Vector DB	ChromaDB (persistent)	Long-term memory storage and retrieval
Embeddings	Gemini Embedding API	Text vectorization for memory
Browser Automation	Playwright (Python)	Web scraping, form automation
Task Scheduler	APScheduler	Cron-like task scheduling
Containerization	Docker + Docker Compose	Environment isolation and deployment

Reverse Proxy	Nginx	TLS termination, load balancing
Testing	Pytest + Vitest	Backend and frontend unit/integration tests
Linting/Formatting	Ruff + Black (Python) / ESLint (TS)	Code quality enforcement
Config Management	pydantic-settings	Typed environment variable management

9. Folder Structure

Project Folder Layout
jarvis/
backend/
app/
main.py # FastAPI app entry point
config.py # pydantic-settings config
brain/
intent_classifier.py # Rule + LLM intent detection
task_planner.py # Multi-step task decomposition
orchestrator.py # Agent coordination
response_aggregator.py # Merge multi-agent outputs
agents/
base_agent.py # Abstract base class
chat_agent.py
web_agent.py
file_agent.py
system_agent.py
memory_agent.py
memory/
vector_store.py # ChromaDB wrapper
memory_manager.py # Store/retrieve/expire logic

```
      embeddings.py        # Embedding generation
    tools/
      web_search.py
      file_ops.py
      os_control.py
      browser.py           # Playwright automation
      scheduler.py         # APScheduler integration
    voice/
      stt.py               # Whisper inference
      tts.py               # Coqui TTS inference
      vad.py               # Voice activity detection
    api/
      routes/
        chat.py
        voice.py
        memory.py
        automation.py
      middleware/
        auth.py
        rate_limiter.py
        cors.py
    tests/
      unit/
      integration/
  requirements.txt
  Dockerfile
frontend/
  src/
    components/
      ChatPanel/
```

```
        VoicePanel/
        MemoryBrowser/
        AgentActivity/
        Settings/
      hooks/
        useWebSocket.ts
        useVoice.ts
        useMemory.ts
      stores/              # Zustand state management
      styles/
      tests/
    vite.config.ts
    Dockerfile
  docker/
    docker-compose.yml
    docker-compose.prod.yml
    nginx.conf
  docs/
    architecture.md
    api-reference.md
    setup-guide.md
    agent-development.md
  .env.example
  CHANGELOG.md
  README.md
```

10. API Specification

10.1 REST Endpoints

Method + Path	Description	Auth Required

GET /health	System health check with component status	No
POST /api/v1/chat	Send message, receive full response	JWT
WS /api/v1/chat/stream	WebSocket streaming chat	JWT
POST /api/v1/voice/transcribe	Upload audio, receive transcription	JWT
POST /api/v1/voice/synthesize	Text to audio stream	JWT
GET /api/v1/memory	List all memory entries (paginated)	JWT
POST /api/v1/memory	Create new memory entry	JWT
DELETE /api/v1/memory/{id}	Delete specific memory entry	JWT
DELETE /api/v1/memory/all	Purge all user memories	JWT
GET /api/v1/agents	List registered agents and status	JWT
POST /api/v1/automation/execute	Execute automation command	JWT
GET /api/v1/automation/audit	Retrieve audit log entries	JWT
POST /api/v1/automation/schedule	Create scheduled task	JWT
DELETE /api/v1/automation/kill	Kill all running automation	JWT

11. Data Privacy & Compliance

11.1 Data Storage Policy

- All memory data stored locally on user's machine — no cloud sync by default
- ChromaDB data directory configurable; supports external encrypted volume
- Conversation logs retained for 30 days by default (configurable, can disable)
- Voice audio discarded immediately after STT transcription
- No usage analytics or telemetry collected without explicit opt-in

11.2 User Data Rights

- Right to access: GET /api/v1/memory returns all stored user data
- Right to delete: single command purges all memories and logs
- Right to export: full memory export to JSON on demand
- Right to transparency: audit log shows every action JARVIS has taken

11.3 Security of Sensitive Data

- API keys stored in .env only; never logged, never exposed in API responses
- Memory store encryption at rest (AES-256) in production configuration
- HTTPS required in production; HTTP access logs warn of insecure connections

12. Development Roadmap

Week / Sprint	Deliverables
Week 1: Foundation	FastAPI setup, Gemini API integration, streaming endpoint, .env config, logging, health check, basic tests
Week 2: Frontend	React + Vite scaffold, WebSocket client, streaming chat UI, dark/light theme, mobile responsive layout
Week 3: Multi-Agent Brain	Intent classifier, agent registry, 5 core agents, task planner, orchestrator, agent debug panel in UI
Week 4: Voice + Memory	Whisper STT, Coqui TTS, VAD, voice UI panel, ChromaDB setup, embedding pipeline, RAG integration
Week 5: Automation	File ops, browser automation (Playwright), scheduler (APScheduler), reminder system, audit logging, kill switch
Week 6: Testing + Hardening	Unit tests (80% coverage), integration tests, performance benchmarks, security audit, Docker production build, documentation

13. Risk Analysis & Mitigation

Risk	Likelihood / Impact	Mitigation
Gemini API rate limits / outages	Medium / High	Retry with backoff; cached fallback responses; consider Ollama local fallback
Whisper inference too slow on CPU	High / High	Default to whisper-tiny; GPU acceleration path; async queuing for voice
ChromaDB corruption on hard shutdown	Low / High	WAL mode enabled; daily backup cron; data export on shutdown
Automation sandbox escape	Low / Critical	Strict allowlist; dry-run mode; audit log; kill switch; no sudo access

Browser automation detection / blocking	Medium / Medium	Playwright stealth mode; configurable delays; respect robots.txt
Context window overflow (long sessions)	Medium / Medium	Conversation summarization at 80% token limit; sliding window strategy
Multi-agent race conditions	Medium / Medium	asyncio.gather with locks; per-agent timeout; idempotent agent design
Memory relevance decay over time	Low / Low	Relevance scoring; TTL-based expiry; configurable retention policies

14. Success Metrics & KPIs

Metric	Target
Intent routing accuracy	> 90% correct agent selection
Chat response latency (p95)	< 2 seconds
Voice round-trip latency	< 3 seconds
STT word accuracy	> 95% on clean speech
Memory cross-session recall	> 85% relevance on retrieval
Automation command safety	0 unauthorized file system operations
Test coverage (backend)	> 80% unit, > 60% integration
WebSocket concurrent connections	50 without degradation
System uptime (local)	> 99.5%
Developer setup time	< 30 minutes from clone to running

15. Glossary

Term	Definition
JARVIS	Just A Rather Very Intelligent System — the project codename
LLM	Large Language Model — AI model for text generation (Gemini)
Agent	Specialized AI module responsible for a specific domain of tasks
RAG	Retrieval Augmented Generation — injecting retrieved memory into LLM context

STT	Speech-to-Text — converting audio to text (via Whisper)
TTS	Text-to-Speech — converting text to audio (via Coqui TTS)
VAD	Voice Activity Detection — detecting when a user starts/stops speaking
ChromaDB	Open-source vector database used for persistent memory storage
Intent	The detected purpose behind a user message (e.g., file operation, web search)
Orchestrator	The component that coordinates parallel/sequential agent execution
Embedding	Numerical vector representation of text for semantic similarity search
DXA	Document XML Attribute units — 1 inch = 1440 DXA (used in DOCX formatting)
JWT	JSON Web Token — standard for secure API authentication
CORS	Cross-Origin Resource Sharing — browser security policy for API access
Dry-run	Preview mode — simulates a command without executing destructive operations

16. Document Revision History

Version	Changes
v1.0 — Initial	Original PRD with basic phase structure and feature list
v2.0 — Comprehensive	Added: user personas, acceptance criteria, API spec, security model, data privacy, risk analysis, KPIs, glossary, full folder structure, detailed agent specs

— End of Document —